Testing Policy

Testing Objectives

Testing is done using the Sandwich strategy: 1) Quality checks → 2) Unit tests → 3) Integration test

Quality tests ensure consistency and reliability of all code produced. Unit tests serve to detect errors in program logic or other bugs before leaving development. Integration tests ensure that the system can be deployed correctly and smoothly.

Testing types

Quality tests are done first to catch formatting and style problems. These are quick tests and are to be run locally before commits are sent to the repo.

Unit tests are written concurrently with working code for all functions and modules. Coverage for unit tests are also tracked and tested against.

Finally, integration tests or build verifications are run. These tests start up a container for the backend and frontend to ensure that the application is able to spin up and run. Fully integrated deployment is tested on the staging branch before final server/production deployment

Tools and Environments

The application always exists in a container, from development to deployment. Testing is run within these containers to ensure testing in near identical conditions to deployment.

Sonarqube is deployed on the repo to aid with code quality, reliability, and reducing complexity

The final deployment and its associated tests are run on an AWS server within containers.

Backend

- Quality tests are run using:
  - Black for formatting
  - Ruff for linting
  - MyPy for type checking
  - Bandit for security checks

    - Pylint for dead code and style checks
- Unit tests and coverage
    - Pytest is used to run unit tests and track coverage
    - A HTML coverage report is generated
    - Coverage badges are created
- The backend build is verified usimg a Production startup smoke test within a container

Frontend

- Quality tests and Linting are done using:
    - Prettier for format checking
    - ESLint for linting
- Unit tests and Coverage
    - Vitest runs all tests and tracks duration
    - A detailed coverage report is generated
    - Coverage badges are created
- The frontend is build in its container
    - Vite is used to building the client environment
    - Build artifacts are saved for record keeping

Defect Management Processes

Defects, or failed tests, are caught on pull requests, which are required before any merge and require at least one reviewer to ensure adherence to policies.

Unit tests being written concurrently with code allows for defect management before code is even added to the repo. The CI/CD pipeline simply ensures a set standard of code.

If deployment defects are detected, a rollback strategy is in place to revert to the last functional version and allow revision of code. Roll back is implemented in both the local staging branch deployment and the final production deployment.

Acceptance Criteria

All mentioned tests must pass for a PR to be allowed to be merged.

More details for code quality test can be found in the Coding Standards Document on the GitHub Wiki.

Unit test must all pass and have a total coverage of at least 65%

Integration tests must be able to verify a running built container. On Deployment integration tests a health check must be passed, else a rollback is triggered.